

Course: AI Assisted Coding (23CS002PC304)

Lab 13.2

Name : Samanvith
HTNo : 2303A52090
Batch No : 40

Task 1: Refactoring – Eliminating Repetitive Logic

Task Description:

Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Prompt:

"Refactor the following legacy Python code by encapsulating the repeated area and perimeter computations into a reusable function. Ensure the function has a proper docstring, prints the results, and the overall output remains unchanged."

Code:

```
length = 8

width = 4

print('Area:', length * width)

print('Perimeter:', 2 * (length + width))

length = 12

width = 6

print('Area:', length * width)

print('Perimeter:', 2 * (length + width))"
```

Output:

Area: 32
Perimeter: 24
Area: 72

Perimeter: 36

Task 2: Refactoring – Modularizing Inline Calculations

Task Description:

Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Prompt:

"Refactor this Python script to modularize the inline discount calculations. Extract the logic into a reusable, well-documented function that returns the final amount. Update the script to use this function for amounts 1000 and 2000 with a 10% discount.

Code:

```
amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print('Final Amount:', final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print('Final Amount:', final_amount)"
```

Output:

```
Final Amount: 900.0
Final Amount: 1800.0
```

Task 3: Refactoring – Improving Loop and Conditional

Performance

Prompt:

"Optimize the following Python code that uses inefficient nested loops for lookups. Recommend a more efficient data structure (like a Set) for O(1) lookups, simplify the logic, and include comments explaining the performance improvement.

Code:

```
students = ['Anil', 'Bhavya', 'Charan', 'Divya']
```

```
check_list = ['Charan', 'Esha', 'Bhavya']
```

```
for name in check_list:
```

```
    exists = False
```

```
    for student in students:
```

```
        if name == student:
```

```
            exists = True
```

```
    if exists:
```

```
        print(name, 'found')
```

```
    else:
```

```
        print(name, 'not found')"
```

Output:

```
Charan found
```

```
Esha not found
```

```
Bhavya found
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Prompt:

"Refactor this simple interest calculation code to improve maintainability. Detect the hardcoded numeric values (rate=2, time=5) used directly in the expressions and replace them with descriptive constants defined at the beginning of the script. Update the calculations to use these constants.

Code:

```
print('Simple Interest:', (1000 * 2 * 5) / 100)

print('Simple Interest:', (2000 * 2 * 5) / 100)"
```

Output:

Simple Interest: 100.0
Simple Interest: 200.0

Task 5: Refactoring – Enhancing Variable Naming and Code Clarity

Prompt:

"Refactor the following Python snippet to enhance code clarity. Replace the non-descriptive variables (x, y, z) with meaningful identifiers representing a shopping cart calculation. Add inline comments explaining the logic while ensuring the output remains identical.

Code:

```
x = 50

y = 30

z = x + y

print(z)"
```

Output:

80

